

EE 5239 Nonlinear Optimization: Lecture 11

Matrix Factorization

Mingyi Hong

University Of Minnesota

Outline

- 1 Announcement
- 2 Review of SVD and ED, and Applications
- 3 Application: Page Rank
- 4 The Power Method
- 5 Application: Text Semantic

Plans for the next few weeks

- For HW 5 Problem 3.3.3, only do $N = 2$
- Problem 3.3.3, only do $N = 2$
- We will start discussing different “topics” in optimization and applications

Outline

- 1 Announcement
- 2 Review of SVD and ED, and Applications
- 3 Application: Page Rank
- 4 The Power Method
- 5 Application: Text Semantic

This Time

- Introduce learning and optimization methods based on matrix decomposition/factorization
- Here the matrix is usually the “data matrix”, can be huge
- Decomposing the matrix can extract **key information** about the data
- Main Applications:
 - ① Graph clustering
 - ② Document analysis
 - ③ Page rank
- Main Algorithm: The Power Method

The Eigen Decomposition

- ① Eigenvalue λ for a **square matrix**: $\mathbf{A}\mathbf{x} = \lambda\mathbf{x}$ for some $\mathbf{x} \neq 0$.
- ② Spectral radius: $\rho(\mathbf{A}) = \max_k \{|\lambda_k|\}$, λ_k is an eigenvalue of $\mathbf{A}$.
- ③ Eigen Decomposition (ED) for a **symmetric matrix** $\mathbf{A} \in \mathbb{R}^{K \times K}$:

$$\mathbf{A} = \mathbf{X}\mathbf{\Lambda}\mathbf{X}'$$

$\mathbf{X}$ is orthonormal, $\mathbf{\Lambda}$ is diagonal and real, contains elements $\lambda_1, \dots, \lambda_K$

- ④ $\mathbf{A} \succeq 0$ iff $\lambda_k \geq 0$ for all k
- ⑤ Given an ED, one can express $\mathbf{A} \in \mathbb{R}^{K \times K}$ as

$$\mathbf{A} = \sum_{k=1}^K \lambda_k \mathbf{x}_k \mathbf{x}_k^T$$

where $\mathbf{x}_k$'s are the eigenvectors

The SVD

- ① Singular value decomposition (SVD):

$$\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}' \in \mathbb{R}^{m \times n},$$

where $\mathbf{\Sigma} \in \mathbb{R}^{m \times n}$ rectangular diagonal matrix with diagonals $\sigma_1 \geq \sigma_2 \geq \cdots, \sigma_n \geq 0$; $\mathbf{U} \in \mathbb{R}^{m \times m}$, $\mathbf{V} \in \mathbb{R}^{n \times n}$ orthonormal

- ② Relationship of SVD and ED: σ_i^2 is an eigenvalue of $\mathbf{A}\mathbf{A}'$ Why?

$$\mathbf{A}\mathbf{A}' = (\mathbf{U}\mathbf{\Sigma}\mathbf{V}')(\mathbf{V}\mathbf{\Sigma}\mathbf{U}') = \mathbf{U}\mathbf{\Sigma}^2\mathbf{U}'$$

The Principal Component Analysis

- Useful tool in statistical analysis
- Obtain low dimensional representation of the data
- Convert observations of correlated variables to **linearly uncorrelated** variables

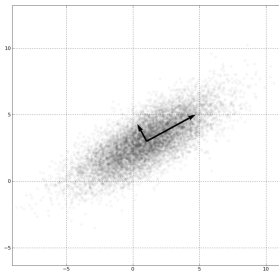


Figure 0.1: PCA for a set of 2 2-D Gaussian random variables [Wikipedia].

The Principal Component Analysis: Details

- Given a dataset $\{\mathbf{a}_1, \dots, \mathbf{a}_M\}$, with each $\mathbf{a}_i \in \mathbb{R}^K$
- An optimization formulations of the PCA, linking PCA with ED

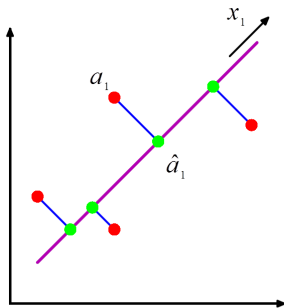


Figure 0.2: Illustration of PCA.

The PCA: Variance Maximization

- We have the data matrix $\mathbf{A} \in \mathbb{R}^{M \times K}$
- A total of M data points, each of dimension K
- Get a low dimensional representation – the simplest case with a single vector to represent the entire data set
- **The objective:** Find a vector $\mathbf{x}_1 \in \mathbb{R}^K$ such that
 - ① We compute inner product of each data $\mathbf{a}_i$ on: $\mathbf{x}_1^T \mathbf{a}_i$
 - ② The resulting inner product is relatively large (keeps the information from the original data as much as possible)

The PCA: Variance Maximization

- **The objective:** Find a vector $\mathbf{x}_1 \in \mathbb{R}^K$, so its $\mathbf{x}_1^T \mathbf{a}_i$ is large for each data set

The PCA: Variance Maximization

- **The objective:** Find a vector $\mathbf{x}_1 \in \mathbb{R}^K$, so its $\mathbf{x}_1^T \mathbf{a}_i$ is large for each data set
- Sample mean of the **projected** data point is $\mathbf{x}_1^T \bar{\mathbf{a}}$ with

$$\bar{\mathbf{a}} = \frac{1}{M} \sum_{i=1}^M \mathbf{a}_i$$

The PCA: Variance Maximization

- **The objective:** Find a vector $\mathbf{x}_1 \in \mathbb{R}^K$, so its $\mathbf{x}_1^T \mathbf{a}_i$ is large for each data set
- Sample mean of the **projected** data point is $\mathbf{x}_1^T \bar{\mathbf{a}}$ with

$$\bar{\mathbf{a}} = \frac{1}{M} \sum_{i=1}^M \mathbf{a}_i$$

- Sample variance of the **projected** data

$$\frac{1}{M} \sum_{i=1}^M (\mathbf{x}_1^T \mathbf{a}_i - \mathbf{x}_1^T \bar{\mathbf{a}})(\mathbf{x}_1^T \mathbf{a}_i - \mathbf{x}_1^T \bar{\mathbf{a}})^T = \mathbf{x}_1^T \mathbf{S} \mathbf{x}_1$$

where

$$\mathbf{S} = \frac{1}{M} \sum_{i=1}^M (\mathbf{a}_i - \bar{\mathbf{a}})(\mathbf{a}_i - \bar{\mathbf{a}})^T$$

The PCA: Variance Maximization

$$\max_x \mathbf{x}^T \mathbf{S} \mathbf{x}, \quad \text{s.t. } \|\mathbf{x}\|^2 = 1$$

- Maximize $\mathbf{x}_1^T \mathbf{S} \mathbf{x}_1$ w.r.t. $\mathbf{x}_1$
- Constrain the size of $\mathbf{x}_1$, so add an normalization $\|\mathbf{x}_1\| = 1$

The PCA: Variance Maximization

$$\max_{\mathbf{x}} \mathbf{x}^T \mathbf{S} \mathbf{x}, \quad \text{s.t. } \|\mathbf{x}\|^2 = 1$$

- Maximize $\mathbf{x}_1^T \mathbf{S} \mathbf{x}_1$ w.r.t. $\mathbf{x}_1$
- Constrain the size of $\mathbf{x}_1$, so add an normalization $\|\mathbf{x}_1\| = 1$
- Let's use the KKT condition to solve it; Lagrangian

$$L(\mathbf{x}_1, \lambda_1) = \mathbf{x}_1^T \mathbf{S} \mathbf{x}_1 + \lambda_1(1 - \|\mathbf{x}_1\|^2)$$

- Take derivative

$$2\mathbf{S}\mathbf{x}_1 = 2\lambda_1\mathbf{x}_1$$

The PCA: Variance Maximization

$$\max_{\mathbf{x}} \mathbf{x}^T \mathbf{S} \mathbf{x}, \quad \text{s.t. } \|\mathbf{x}\|^2 = 1$$

- Maximize $\mathbf{x}_1^T \mathbf{S} \mathbf{x}_1$ w.r.t. $\mathbf{x}_1$
- Constrain the size of $\mathbf{x}_1$, so add an normalization $\|\mathbf{x}_1\| = 1$
- Let's use the KKT condition to solve it; Lagrangian

$$L(\mathbf{x}_1, \lambda_1) = \mathbf{x}_1^T \mathbf{S} \mathbf{x}_1 + \lambda_1(1 - \|\mathbf{x}_1\|^2)$$

- Take derivative

$$2\mathbf{S}\mathbf{x}_1 = 2\lambda_1\mathbf{x}_1$$

- $\mathbf{x}_1$ must be the **largest** eigenvalue of $\mathbf{S}$ (k.a.k a principal component)

$$\mathbf{x}_1^T \mathbf{S} \mathbf{x}_1 = \lambda_1$$

The PCA: Variance Maximization

- The subsequent principal components must be orthogonal to x_1
- They are the second, third, ... eigenvectors

The PCA: Variance Maximization

- The subsequent principal components must be **orthogonal** to x_1
- They are the second, third, ... eigenvectors
- PCA problem = ED on the **variance of the data**
- Finds a direction that **explains as much variance** as possible in the data

Outline

- 1 Announcement
- 2 Review of SVD and ED, and Applications
- 3 Application: Page Rank
- 4 The Power Method
- 5 Application: Text Semantic

The Page Rank Problem

- Determine where google displays a web page
- Need to rank the pages

The Page Rank Problem

- Determine where google displays a web page
- Need to rank the pages
- **Key Assumption:** A page has high rank if many high rank pages are linked to it
- Google PageRank is based on the link structure of the web graph

The Page Rank Problem

- Determine where google displays a web page
- Need to rank the pages
- **Key Assumption:** A page has high rank if many high rank pages are linked to it
- Google PageRank is based on the link structure of the web graph
- It does not depend on the the contents
- After performing PageRank, Hypertext Analysis needs to be performed based on page contents to actually rank each query (not discussed here)

The Page Rank Problem

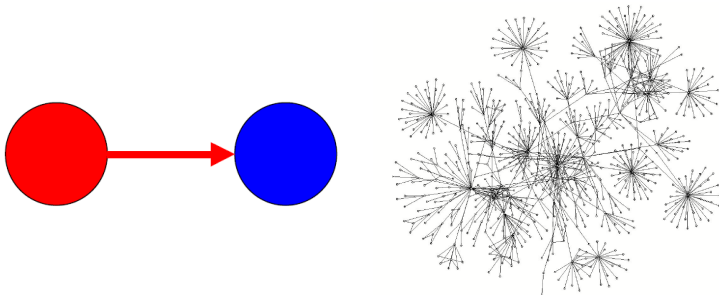
- Determine where google displays a web page
- Need to rank the pages
- **Key Assumption:** A page has high rank if many high rank pages are linked to it
- Google PageRank is based on the link structure of the web graph
- It does not depend on the the contents
- After performing PageRank, Hypertext Analysis needs to be performed based on page contents to actually rank each query (not discussed here)
- **Interpretation:** If one randomly clicks the links, what is the probability that one will be at a given page

How to do PageRank?

- Construct a huge graph of web pages
- Represent the graph by certain **matrix**
- Transform the matrix to the **“Google Matrix”**
- Do **PCA** on the Google matrix
- For more discussion on how this works, see [Here](#)

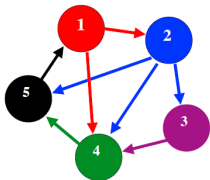
Step 1: Graph Construction

- Make the webpages as nodes in the graph
- Make the links as edges of the graph



Step 2: Construct a Web Matrix

- Convert the graph to a matrix S
- A non-zero element in the matrix corresponds to a link



$$S = \begin{pmatrix} 0 & \frac{1}{2} & 0 & \frac{1}{2} & 0 \\ 0 & 0 & \frac{1}{3} & \frac{1}{3} & \frac{1}{3} \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 \end{pmatrix}$$

Figure 0.1: Constructing a Web Matrix [I. Ipsen 06].

Step 2: Construct a Web Matrix

- Row i of $\mathbf{S}$: links from page i to other pages
- Column i of $\mathbf{S}$: links into page i

Step 2: Construct a Web Matrix

- Row i of $\mathbf{S}$: links from page i to other pages
- Column i of $\mathbf{S}$: links into page i
- Is the matrix symmetric?

Step 2: Construct a Web Matrix

- Row i of $\mathbf{S}$: links from page i to other pages
- Column i of $\mathbf{S}$: links into page i
- Is the matrix symmetric?
- $\mathbf{S}$ is constructed as a stochastic matrix

Step 2: Construct a Web Matrix

- Row i of $\mathbf{S}$: links from page i to other pages
- Column i of $\mathbf{S}$: links into page i
- Is the matrix symmetric?
- $\mathbf{S}$ is constructed as a stochastic matrix
 - 1 All elements in $[0, 1]$
 - 2 Elements in each row sum to 1

Step 3: Google Matrix

- Construct the **google matrix**

$$\mathbf{G} = \alpha \mathbf{S} + (1 - \alpha) \mathbf{1} \mathbf{v}^T$$

- Make sure the graph is **connected**
- Damping factor $0 \leq \alpha \leq 1$
- $\mathbf{1}$ is a column vector of all 1's
- $\mathbf{v}$ is a vector

$$\mathbf{v} \geq 0, \quad \|\mathbf{v}\|_1 = \sum_{i=1}^M |v_i| = 1$$

- Can take $\mathbf{v} = \mathbf{1} \frac{1}{M}$ (M being the size of $\mathbf{G}$)

Step 3: Google Matrix

- One can show that $\mathbf{G}$ is also stochastic
- The google matrix has now a **unique** dominant left eigenvector (due to the strictly positive entries of the matrix)

$$\boldsymbol{\pi}^T \mathbf{G} = \boldsymbol{\pi}^T, \quad |\boldsymbol{\pi}| = 1.$$

- All components of $\boldsymbol{\pi}$ are positive
- This is by Perron-Frobenius Theorem ($\mathbf{G}$ has positive entries)
- π_i is the **PageRank** of web page i
- Again compute the PCA (on $\mathbf{G}^T$)!

Summary of PageRank

- Construct a huge graph of web pages
- Represent the graph by certain **matrix**
- Transform the matrix to the **“Google Matrix”**
- Do **PCA** on the Google matrix
- For more discussion on how this works, see [Here](#)

Outline

- 1 Announcement
- 2 Review of SVD and ED, and Applications
- 3 Application: Page Rank
- 4 The Power Method
- 5 Application: Text Semantic

The Power Method: Algorithm

- The **Power Method** is a very popular method for computing the **largest eigenvalue/eigenvector** of a matrix
- Used by PageRank Algorithm

The Power Method: Algorithm

- The **Power Method** is a very popular method for computing the **largest eigenvalue/eigenvector** of a matrix
- Used by PageRank Algorithm
- Given a matrix $\mathbf{B} \in \mathbb{R}^{K \times K}$
- Algorithm is very simple

① Start with a random vector $\mathbf{x} \in \mathbb{R}^K$

② For $r = 1 \dots, R$

$$\mathbf{x}^{(r+1)} = \mathbf{B}\mathbf{x}^{(r)}, \text{ or } \mathbf{x}^{(r+1)} = \mathbf{B}\mathbf{x}^{(r)} / \|\mathbf{B}\mathbf{x}^{(r)}\|_2$$

③ After R iterations, perform the normalization

$$\mathbf{x}^{(R)} = \mathbf{x}^{(R)} / \|\mathbf{x}^{(R)}\|_2, \quad \lambda = (\mathbf{x}^{(R)})^T \mathbf{B} \mathbf{x}^{(R)}$$

The Power Method: Analysis

- The algorithm is very simple
- Each step only involves in vector-matrix product
- Has nice theoretical convergence guarantee
- An interpretation of the algorithm (gradient descent with infinite stepsize!)
- Consider the following formulation

$$\max \mathbf{x}^T \mathbf{B} \mathbf{x}, \quad \text{s.t. } \|\mathbf{x}\|^2 = 1$$

- Gradient step

$$\mathbf{x}^{t+1} = (\mathbf{x}^t + \alpha 2\mathbf{B}\mathbf{x}^t) / \|\mathbf{x}^t + \alpha 2\mathbf{B}\mathbf{x}^t\|_2$$

- Taking $\alpha \rightarrow \infty$ yields the power iteration!

The Power Method: Analysis

- Let's perform the convergence analysis of power iteration
- Basically the algorithm converges **linearly**
- The rate is dependent on the **ratio** $|\lambda_1|/|\lambda_2|$ (the largest size eigenvalue divided by the second largest size eigenvalue)
- **Note:** If $\lambda_1 = \lambda_2$, algorithm may not work!
- For simplicity, consider the version where the normalization is only performed in the **last iteration**

The Power Method: Analysis

- Let us express the initial vector $\mathbf{x}^{(0)}$ and matrix $\mathbf{B}$ by

$$\mathbf{x}^{(0)} = \mathbf{x}_1 u_1 + \mathbf{x}_2 u_2 + \cdots, \mathbf{x}_K u_k \quad (0.1)$$

$$\mathbf{B} = \mathbf{x}_1 \mathbf{x}_1^T \lambda_1 + \mathbf{x}_2 \mathbf{x}_2^T \lambda_2 + \cdots, \mathbf{x}_K \mathbf{x}_K^T \lambda_k \quad (0.2)$$

The Power Method: Analysis

- Let us express the initial vector $\mathbf{x}^{(0)}$ and matrix $\mathbf{B}$ by

$$\mathbf{x}^{(0)} = \mathbf{x}_1 u_1 + \mathbf{x}_2 u_2 + \cdots, \mathbf{x}_K u_K \quad (0.1)$$

$$\mathbf{B} = \mathbf{x}_1 \mathbf{x}_1^T \lambda_1 + \mathbf{x}_2 \mathbf{x}_2^T \lambda_2 + \cdots, \mathbf{x}_K \mathbf{x}_K^T \lambda_K \quad (0.2)$$

- After the first iteration

$$\mathbf{x}^{(1)} = \mathbf{B} \mathbf{x}^{(0)} = \mathbf{x}_1 \lambda_1 u_1 + \mathbf{x}_2 \lambda_2 u_2 + \cdots, \mathbf{x}_K \lambda_K u_K \quad (0.3)$$

The Power Method: Analysis

- Let us express the initial vector $\mathbf{x}^{(0)}$ and matrix $\mathbf{B}$ by

$$\mathbf{x}^{(0)} = \mathbf{x}_1 u_1 + \mathbf{x}_2 u_2 + \cdots, \mathbf{x}_K u_K \quad (0.1)$$

$$\mathbf{B} = \mathbf{x}_1 \mathbf{x}_1^T \lambda_1 + \mathbf{x}_2 \mathbf{x}_2^T \lambda_2 + \cdots, \mathbf{x}_K \mathbf{x}_K^T \lambda_K \quad (0.2)$$

- After the first iteration

$$\mathbf{x}^{(1)} = \mathbf{B}\mathbf{x}^{(0)} = \mathbf{x}_1 \lambda_1 u_1 + \mathbf{x}_2 \lambda_2 u_2 + \cdots, \mathbf{x}_K \lambda_K u_K \quad (0.3)$$

- After r iterations

$$\mathbf{x}^{(r)} = \mathbf{x}_1 \lambda_1^r u_1 + \mathbf{x}_2 \lambda_2^r u_2 + \cdots, \mathbf{x}_K \lambda_K^r u_K \quad (0.4)$$

The Power Method: Analysis

- If $u_1 \neq 0$ (easy to satisfy by random initialization)

$$\mathbf{x}^{(r)} \propto u_1 \lambda_1^r \left(\mathbf{x}_1 + \underbrace{\sum_{j=2}^K \frac{u_j}{u_1} \left(\frac{\lambda_j}{\lambda_1} \right)^r \mathbf{x}_j}_{\text{error}} \right) \quad (0.5)$$

The Power Method: Analysis

- If $u_1 \neq 0$ (easy to satisfy by random initialization)

$$\mathbf{x}^{(r)} \propto u_1 \lambda_1^r \left(\mathbf{x}_1 + \underbrace{\sum_{j=2}^K \frac{u_j}{u_1} \left(\frac{\lambda_j}{\lambda_1} \right)^r \mathbf{x}_j}_{\text{error}} \right) \quad (0.5)$$

- So the term inside $(\cdot)$ converges to $\mathbf{x}_1$ **linearly**, with the constant proportional to $|\lambda_2/\lambda_1|^r$

The Power Method: Analysis

- What if you want to find the “second” largest eigenvalue/vector?

The Power Method: Analysis

- What if you want to find the “second” largest eigenvalue/vector?
- Perform the **Gram-Schmidt orthogonalization**

The Power Method: Analysis

- What if you want to find the “second” largest eigenvalue/vector?
- Perform the **Gram-Schmidt orthogonalization**
- Suppose we already have $\mathbf{x}_1$; compute $\mathbf{x}_2$ with a random initialization $\mathbf{x}_2^{(0)}$
- Perform the following on $\mathbf{x}_2^{(0)}$

$$\mathbf{x}_2^{(0)} = \mathbf{x}_2^{(0)} - \mathbf{x}_1^T \mathbf{x}_2^{(0)} \mathbf{x}_1 \quad (0.6)$$

The Power Method: Analysis

- What if you want to find the “second” largest eigenvalue/vector?
- Perform the **Gram-Schmidt orthogonalization**
- Suppose we already have $\mathbf{x}_1$; compute $\mathbf{x}_2$ with a random initialization $\mathbf{x}_2^{(0)}$
- Perform the following on $\mathbf{x}_2^{(0)}$

$$\mathbf{x}_2^{(0)} = \mathbf{x}_2^{(0)} - \mathbf{x}_1^T \mathbf{x}_2^{(0)} \mathbf{x}_1 \quad (0.6)$$

- Verify that: $\mathbf{x}_1^T \mathbf{x}_2^{(0)} = 0$
- Verify that when performing power iteration on the new $\mathbf{x}_2^{(0)}$

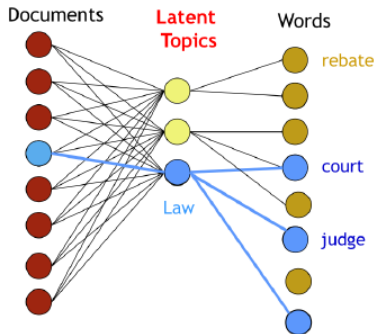
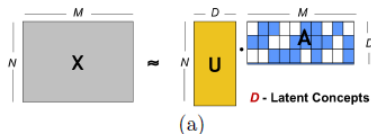
$$\mathbf{x}_1^T \mathbf{x}_2^{(r)} = 0, \forall r$$

Outline

- 1 Announcement
- 2 Review of SVD and ED, and Applications
- 3 Application: Page Rank
- 4 The Power Method
- 5 Application: Text Semantic

PCA Apps: Text Semantic Analysis

- 1 Identifying the semantic meaning of documents
- 2 Documents clustering
- 3 Trend identification/analysis



PCA Apps: Text Semantic Analysis

- **Dataset:** Given M docs with words from a vocabulary of size K
- **Objective:** Discover **hidden topics**, represent documents semantically
- The hidden topics are our “principal component space”

PCA Apps: Text Semantic Analysis

- **Dataset:** Given M docs with words from a vocabulary of size K
- **Objective:** Discover **hidden topics**, represent documents semantically
- The hidden topics are our “principal component space”
- **Data Preprocessing:** Arrange the data into a big matrix $\mathbf{A}$
 - 1 **Bag-of-words** data model
 - 2 $\mathbf{A}$ is a large $M \times K$ matrix, M the number of documents, K the number of features (words)
 - 3 The elements represents **how many times** a word appears in a document
 - 4 Each row of $\mathbf{A}$ is a data point, which is a very **sparse** vector

PCA Apps: Text Semantic Analysis

- **Data Preprocessing:** Arrange the data into a big matrix $\mathbf{A}$
 - ① **Bag-of-words** data model
 - ② $\mathbf{A}$ is a large $M \times K$ matrix, M the number of documents, K the number of features (words)
 - ③ The elements represents **how many times** a word appears in a document
 - ④ Each row of $\mathbf{A}$ is a data point, which is a very **sparse** vector
- Perform the **approximate SVD** on $\mathbf{A}^T$

$$\mathbf{A}^T \approx \mathbf{U}\mathbf{\Sigma}\mathbf{V}$$

where $\mathbf{U} \in \mathbb{R}^{K \times P}$, $\mathbf{\Sigma} \in \mathbb{R}^{P \times P}$, $\mathbf{V} \in \mathbb{R}^{P \times M}$

- Find the **largest P** singular values and singular vectors
- Top P **left singular vectors** represents topics – **Semantic space**

PCA Apps: Text Semantic Analysis (cont.)

- A single column in $\mathbf{A}^T$, say $\mathbf{a}_i$ is a single document. We have

$$\mathbf{a}_i \approx \mathbf{U}\Sigma\mathbf{v}_i := \mathbf{U}\mathbf{c}_i = \sum_{p=1}^P \mathbf{u}_p c_{i,p}$$

where $\mathbf{v}_i$ is the i th column in $\mathbf{V}$; $\mathbf{u}_p$ is the p th column of $\mathbf{U}$

- Each column of $\mathbf{U}$ represents some topics
- View each document as a linear combination of “topics”

Relationship to PCA

- Recall finding **left** singular values of $\mathbf{A}^T$ is **equivalent** to finding eigenvectors for $\mathbf{A}^T \mathbf{A}$
- Suppose we want the first semantic space (the most popular trend)
- The problem becomes

$$\mathbf{A}^T \mathbf{A} \approx \lambda_1 \mathbf{x}_1 \mathbf{x}_1^T$$

- Interpretation by variance maximization?
- The **locations** of large elements of $\mathbf{x}_1$ represent important terms
- The subsequent semantic vectors can be found by computing other eigenvectors

Relationship to PCA

- Optimization formulation

$$\begin{aligned} \min_{\mathbf{x}_1} \quad & \|\mathbf{A}\mathbf{A}^T - \mathbf{x}_1\mathbf{x}_1^T\|^2 \\ \text{subject to} \quad & \|\mathbf{x}_1\|^2 = 1 \end{aligned}$$

- Expand the objective, utilize $\|\mathbf{x}_1\|^2 = 1$, it is equivalent to

$$\begin{aligned} \max_{\mathbf{x}_1} \quad & \mathbf{x}_1^T \mathbf{A}\mathbf{A}^T \mathbf{x}_1 \\ \text{subject to} \quad & \|\mathbf{x}_1\|^2 = 1 \end{aligned}$$

- The problem size can be huge. For example, a text data set for NYTimes contains 300,000 articles and has a dictionary of 102,660 unique words
- In practice, only care about a few top topics

Relationship to PCA

Table 1: Words associated with the top 5 sparse principal components in NYTimes

1st PC (6 words)	2nd PC (5 words)	3rd PC (5 words)	4th PC (4 words)	5th PC (4 words)
million	point	official	president	school
percent	play	government	campaign	program
business	team	united.states	bush	children
company	season	u.s	administration	student
market	game	attack		
companies				

Figure 0.1: Semantic Analysis for NYTimes Documents [Zhang and Ghaoui 11]